

Trading on Text, Pricing by Approximation

A Machine Learning Study

1 Backtesting trading strategies based on sentiment indicators

We design two trading rules driven by news sentiment, both distinct from the moving-average crossover (MAcross) and the RSI strategy of Lab 6, and evaluate them after a 1% transaction cost on every trade. The first is a single-stock rule applied to HSBC and JPM separately and judged against single-stock buy-and-hold, which is the most literal reading of the assignment. The second is a relative-sentiment pairs trade between the two banks, judged against an equal-weight long-both portfolio. All scoring uses the Lab 6 **performance** harness, and all tuning is done on a training slice with the held-out slice kept aside, so the headline numbers are not the product of in-sample fitting alone.

1.1 Data and sentiment indicators

We use the daily HSBC and JPM news files. After aligning the two series on their common dates we have 602 trading days from 2018-01-02 to 2020-05-22, with identical date indexes and no missing values once the sentiment series are linearly interpolated and end-filled. The price target is the adjusted close, and $r_t = P_t/P_{t-1} - 1$ is the daily simple return.

From the part-of-speech scores in each file we build six sentiment indicators ev and keep the relative news-volume series RVT:

$$\begin{aligned} \text{BULL} &= 0.33 (\text{positiveP} + \text{certaintyP} + \text{finupP}), \\ \text{BEAR} &= 0.33 (\text{negativeP} + \text{uncertaintyP} + \text{findownP}), \\ \text{BBr} &= 100 \cdot \frac{\text{BULL}}{\text{BULL} + \text{BEAR}}, \quad \text{PNlog} = 0.5 \log \frac{\text{positiveP} + 1}{\text{negativeP} + 1}, \end{aligned}$$

together with the raw positiveP and negativeP scores, giving the indicator set:

$\{\text{positiveP}, \text{negativeP}, \text{BULL}, \text{BEAR}, \text{BBr}, \text{PNlog}\}$. The sample ends in May 2020, so the COVID crash of February and March 2020 sits inside the data. This matters for interpretation, because the crash is the most volatile stretch of the sample and it lands inside the held-out test slice once the data is split for tuning.

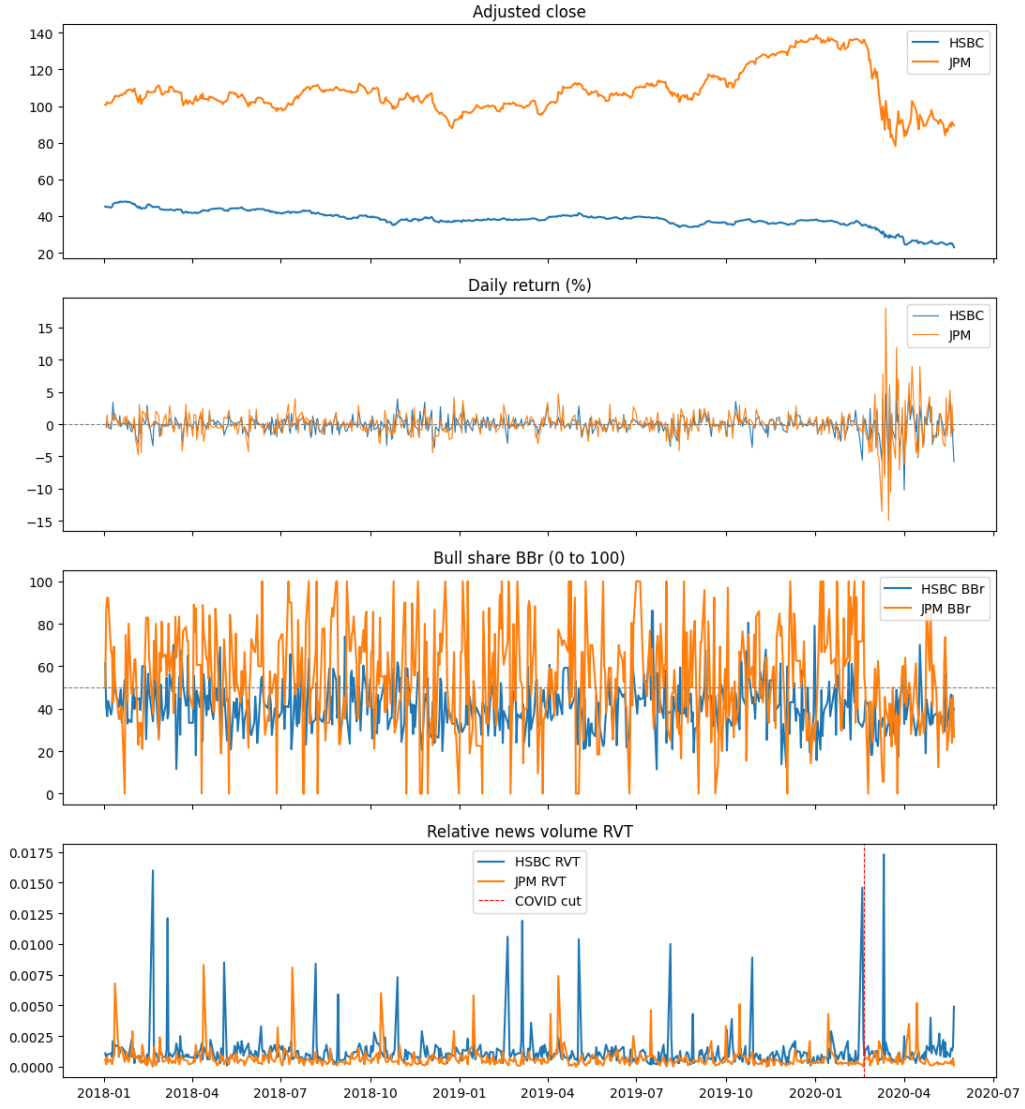


Figure 1: Adjusted close, daily return in %, bull share BBr, and relative news volume RVT for HSBC and JPM. RVT spikes during the COVID weeks, which is what the regime filter is built to catch.

1.2 Strategy design

1.2.1 Single-stock sentiment z-score band

We standardise an indicator over a rolling window of n days,

$$z_t = \frac{ev_t - \mu_t}{\sigma_t}, \quad \mu_t = \frac{1}{n} \sum_{i=t-n+1}^t ev_i, \quad \sigma_t = \text{sd}(ev_{t-n+1:t}),$$

and take a position when the z-score leaves a band of width k . In momentum mode the raw position is

$$p_t = \begin{cases} +1 & z_t > k, \\ s & z_t < -k, \\ p_{t-1} & \text{otherwise,} \end{cases} \quad s = \begin{cases} 0 & \text{long-only,} \\ -1 & \text{long-short,} \end{cases}$$

and contrarian mode swaps the first two branches, so it fades extremes instead of chasing them. Between the bands the position is held, which makes the rule stateful and keeps turnover low. This differs from MACross, which compares two moving averages of price, and from RSI, which is a bounded oscillator with fixed 30 and 70 levels: here the trigger levels are adaptive and move with the recent mean and spread of the sentiment series itself.

We add a regime filter on news volume. We smooth RVT over 20 days and compare it to its own expanding (backward-looking) quantile q ; when smoothed volume is in its top tail the rule stands aside ($p_t = 0$). The filter uses a smoothed series and a high quantile so that it switches on for contiguous stretches such as the COVID news storm rather than toggling day to day, which would add turnover and be punished by the cost.

Every position is shifted forward one day, $\tilde{p}_t = p_{t-1}$, so a trade is only taken the day after the signal is known. The strategy return is $r_t^{me} = \tilde{p}_t r_t$.

1.2.2 Relative-sentiment pairs trade

The pairs rule trades the gap between the two banks' sentiment, $g_t = ev_t^{\text{JPM}} - ev_t^{\text{HSBC}}$. We standardise g_t exactly as above and take $\tilde{p}_t \in \{-1, 0, +1\}$ from the same band rule. The strategy return is the spread,

$$r_t^{me} = \tilde{p}_t (r_t^{\text{JPM}} - r_t^{\text{HSBC}}),$$

so when JPM sentiment is richer than HSBC we are long JPM and short HSBC, and the reverse when it is poorer. The construction is dollar-neutral by design (equal size on each leg). Because there is no single-stock buy-and-hold to compare against, the benchmark is the equal-weight long-both portfolio of Lab 7, $r_t^{bh} = 0.5 r_t^{\text{JPM}} + 0.5 r_t^{\text{HSBC}}$. The signal is a cross-sectional comparison between two assets, not a function of one price path, so it too is distinct from MACross and RSI.

One point of terminology, because the word recurs in the results. In the single-stock rule, “long-short” means the position in one stock can be long (+1) or short (−1) of that same stock. In the pairs rule, “long-short” means being long one bank and short the other at the same time, the dollar-neutral construction just described. The Side column in Table 1

therefore carries a different meaning across the two panels. The pairs long-only variant, where it appears, holds only the leg the signal points to and sits in cash otherwise, so it is read as one leg at a time rather than a true flat-or-long rule.

1.3 Backtesting methodology

Scoring. We keep the Lab 6 **performance** routine and report all eight of its metrics: cumulative return, annual return, annualised Sharpe ratio, win %, annualised volatility, maximum drawdown, maximum drawdown length, and the number of trades. Cumulative return follows the lab convention of an additive running sum of daily returns minus a one-off charge of $N_{tr} c$, where $c = 0.01$ and N_{tr} is the trade count (sign changes in the position, plus one). The pairs trade charges two legs per rebalance. So a reported excess of, say, 1.14 means the strategy’s additive cumulative return exceeds the benchmark’s by 1.14 after cost.

Three properties of the lab formulas are kept faithfully and should be read with care. The additive (not compounded) cumulative return returns **NaN** for the annual return and Sharpe of any strategy that loses more than 100% on this basis. The annualised Sharpe divides an already-annualised return by an annualised volatility, so its magnitudes are large and should be read only as a ranking under one fixed formula. The maximum-drawdown-length column is near-always zero by construction, so we treat maximum drawdown as the risk figure and ignore the length.

Tuning without look-ahead. We split the sample chronologically at 70% (the cut falls in September 2019, so the held-out slice contains the crash), tune on the earlier part only, and report on both the full sample and the held-out part. For every stock, indicator, mode, and direction we grid-search the window $n \in \{10, 20, 40, 60, 80\}$, the threshold $k \in \{0.5, 1.0, 1.5, 2.0, 2.5\}$, and the filter quantile $q \in \{\text{off}, 0.85, 0.90\}$, with in-sample excess over the benchmark as the objective. The grid is small enough to score exhaustively, so the search returns the true optimum; a simulated-annealing run (the Lab 7 method) lands on a comparable point and is shown for completeness. A configuration that is positive on the training slice, the full sample, and the held-out slice at once is far more credible than one that is positive only over the full sample.

Cost is the binding constraint. A MAcross run on the BEAR series with the 1% cost makes the point: it trades about 80 times and only just clears buy-and-hold on HSBC (excess +0.09) while losing heavily on JPM (excess −0.92). With a 1% charge on every trade, turnover is the enemy, so the designs above hold a position until the signal clearly flips.

1.4 Results

Configurations that beat the benchmark. Table 1 lists the strongest configurations whose excess over the benchmark is positive after cost, for both designs. The single-stock winners are one rule family: HSBC in contrarian mode, long-short, with the news filter on, fading sentiment extremes. The same rule clears buy-and-hold on positiveP, BULL, and BBr, with full-sample excess between 0.98 and 1.14 and trade counts in the teens to low twenties. A separate clean result is the HSBC contrarian long-only rule, which beats buy-and-hold by 0.62 with a single trade. On the pairs side the winner is positiveP in momentum mode, long-short, beating the equal-weight portfolio by 0.98 on twelve trades.

Table 1: Configurations beating the benchmark after a 1% cost. Excess is benchmark-relative additive cumulative return on the training slice, full sample, and held-out test slice. Single-stock rows are scored against single-stock buy-and-hold; pairs rows against the equal-weight long-both portfolio.

Asset	Indicator	Mode	Side	(n, k, q)	Train	Full	Test	Trades
<i>Single-stock band (vs. buy-and-hold)</i>								
HSBC	positiveP	contrarian	long-short	(20, 2.0, 0.85)	0.460	1.142	0.749	12
HSBC	BULL	contrarian	long-short	(10, 2.0, 0.85)	0.482	0.997	0.582	18
HSBC	BBr	contrarian	long-short	(10, 2.0, 0.85)	0.416	0.983	0.474	24
HSBC	PNlog	momentum	long-short	(40, 2.0, 0.85)	0.457	0.981	0.692	19
JPM	PNlog	contrarian	long-short	(40, 2.0, off)	0.233	0.848	0.541	26
HSBC	positiveP	contrarian	long-only	(10, 2.5, off)	0.237	0.623	0.398	1
<i>Pairs spread (vs. equal-weight JPM+HSBC)</i>								
JPM/HSBC	positiveP	momentum	long-short	(20, 2.0, off)	0.310	0.976	0.626	12
JPM/HSBC	BULL	momentum	long-short	(10, 2.5, off)	0.297	0.840	0.242	4
JPM/HSBC	negativeP	momentum	long-short	(20, 2.0, off)	0.312	0.811	0.393	18
JPM/HSBC	BEAR	momentum	long-short	(10, 2.5, off)	0.180	0.722	0.242	4

Headline strategies, full metric set. We take the best held-out-positive row from each table as a headline and report the complete Lab 6 metric set in Table 2. Both headlines trade twelve times against a benchmark that trades twice, both lift the win rate above the benchmark, and both carry a lower annualised volatility than their benchmark. The single-stock headline turns a buy-and-hold cumulative return of -0.633 into $+0.508$, and the pairs headline turns an equal-weight cumulative of -0.313 into $+0.663$.

Table 2: Full Lab 6 metrics after a 1% cost for the two headline strategies. Left: HSBC positiveP, contrarian, long-short, $(n, k, q) = (20, 2.0, 0.85)$. Right: JPM/HSBC positiveP pairs, momentum, long-short, $(n, k) = (20, 2.0)$. Me is the strategy, BH the relevant benchmark.

Metric	Single-stock headline		Pairs headline	
	Me	BH	Me	BH
Cumulative Return	0.508	-0.633	0.663	-0.313
Annual Return	0.188	-0.344	0.238	-0.146
Annualised Sharpe	265.3	-377.2	246.4	-136.9
Win %	0.532	0.490	0.521	0.498
Annualised Volatility	0.179	0.229	0.243	0.268
Maximum Drawdown	-0.628	-0.181	-0.806	-0.263
Max Length Drawdown	0	0	0	0
n.trades	12	2	12	2

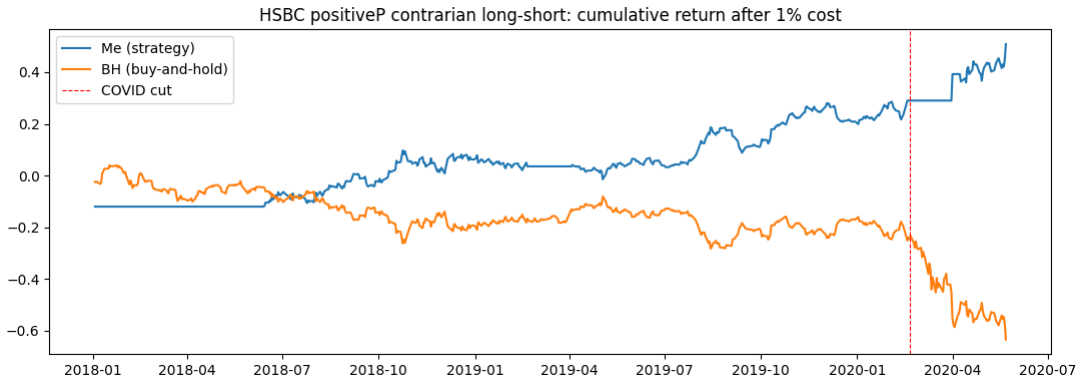


Figure 2: HSBC positiveP contrarian long-short: cumulative return after 1% cost against buy-and-hold. The dashed line marks the COVID cut.

Rolling windows and the cost extension. The rolling test slides a window of 254 days (about one year) and then 127 days (about half a year) across the sample and averages the within-window mean daily return. Table 3 reports the single-stock headline with the cost off (the lab behaviour) and with the cost spread across each window and charged (the extra-credit extension). The strategy stays above the benchmark and stays positive after cost in both window lengths. The per-window edge is small in absolute terms, which is the honest caveat, but because the rule trades only about a dozen times the cost spread over any single window is tiny, so charging it barely moves the average. This is the mirror image of the high-turnover MAcross baseline.

Table 3: Rolling-window average of the within-window mean daily return, single-stock headline. “Cost on” spreads the trade cost across each window (extra credit).

Window	# windows	Me (cost off)	BH (cost off)	Me (cost on)
254	347	+0.00078	-0.00047	+0.00057
127	474	+0.00088	-0.00061	+0.00063

Robustness. Two checks guard against a single lucky setting. First, holding the headline indicator, mode, and direction fixed and sweeping n against k gives a positive excess for every window length once $k \geq 1.5$, with the headline point at $(n, k) = (20, 2.0)$ sitting inside a broad positive plateau rather than on an isolated spike. The negative region at $k = 0.5$ and $k = 1.0$ is expected: a low threshold trips the bands too often and turnover rises. Second, splitting the headline return at the COVID cut shows the edge is not purely a crash effect: the pre-COVID excess over buy-and-hold is 0.630 (over 535 days) against 0.612 during the crash (over 66 days). The crash contributes an outsized amount per day, and the long-short rule clearly benefits from being able to go short into it, but a positive edge is present in the calm part of the sample as well.

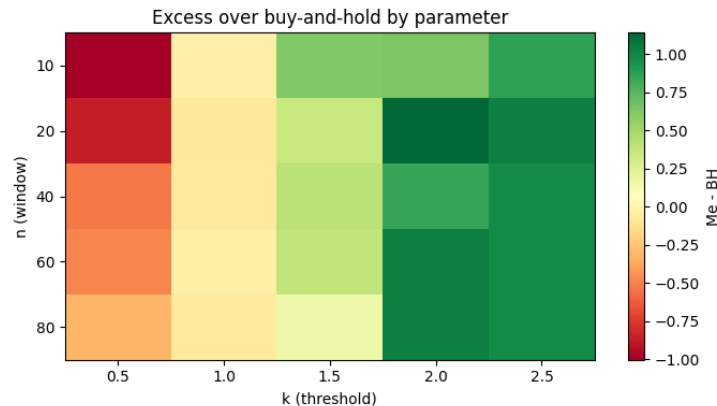


Figure 3: Full-sample excess over buy-and-hold across the window n and threshold k , HSBC positiveP contrarian long-short. The right side of the grid is a stable positive plateau.

1.5 Discussion

Both designs beat their benchmark after the 1% cost on the training slice, the full sample, and the held-out slice that contains the crash, while trading roughly a dozen times. The recurring lesson is that with a 1% charge the surviving strategies are the low-turnover ones: the rule holds a position until the z-score clearly flips, and the news-volume filter is built to switch in blocks rather than day to day, precisely so it does not add trades. The single-stock edge is contrarian on HSBC, fading unusual spikes in positive news rather

than chasing them, and the same rule clears buy-and-hold across three different indicators, which argues against an overfit point.

Three caveats temper the result. HSBC and JPM are large, heavily covered banks, and the sentiment-predicts-returns evidence is generally stronger for smaller, less-covered names, so these two megacaps are a conservative test rather than a favourable one. The tuning objective is in-sample excess, so the full-sample figures flatter the strategies; the held-out and sensitivity results are the parts to lean on. Finally, the large Sharpe magnitudes, the NaN annualisations for blown-up configurations, and the zero drawdown-length column are properties of the Lab 6 formulas rather than results in their own right, and cumulative return is read on the lab's additive, cost-subtracted basis throughout.

2 Machine learning for option valuation

We teach machine learning models the pricing function of two option contracts whose exact valuation has no closed formula and requires slow numerical methods: (a) American put and call options, and (b) barrier down-and-out call options under the Heston stochastic volatility model. Following the assignment, the ground truth for (a) is a Cox-Ross-Rubinstein (CRR) binomial tree and the ground truth for (b) is Monte Carlo simulation of the Heston model with the Euler scheme of Lecture 10. For each problem we generate a synthetic dataset of (parameters, price) pairs with the numerical engine, train Gaussian Process Regression (GPR) and a feed-forward neural network (NN) on identical splits, and evaluate held-out prediction accuracy and per-option computational speedup. The two pipelines are run separately end to end and compared at the close. Everything is seeded and reproducible; the full implementation is in the submitted notebook `ML_PS2_Q2.ipynb`, and the two datasets are stored as (X, y) tables in `dataset_q2_american.csv` and `dataset_q2_barrier.csv`.

The assignment lists five deliverables, and the pipeline below addresses each in turn: choosing the input parameters and justified ranges (Table 4); drawing n random parameter combinations inside the 500 to 2000 band the assignment allows, by seeded Latin hypercube ($n = 1500$ for the American set, $n = 1000$ for the barrier set); pricing each combination with the numerical engine ($N = 1000$ -step tree, or 20,000-path daily-monitored Heston Monte Carlo); storing the result as an (X, y) table; training GPR and a neural network to learn the pricing function; and reporting held-out accuracy together with per-option computational speedup. Each numerical engine is validated against independent references before it is trusted to label any data.

2.1 Setup: surrogate pricing

An option pricer maps contract and model parameters to a present value. When each evaluation needs backward induction over a lattice or thousands of simulated paths, the cost is paid again for every new parameter combination, which is what makes calibration and risk runs expensive. The surrogate idea is to pay that cost once: generate a training table offline with the slow engine, fit a fast regression to it, and from then on price by evaluating the regression in microseconds.

Three design choices apply to both pipelines. First, the spot is fixed at $S_0 = 100$ and moneyness is varied through the strike K : both pricing functions are homogeneous of degree one in money units (the pair (S_0, K) , plus the barrier B for the knock-out contract), so any other spot is a rescaling and including S_0 as a feature would add a redundant dimension. Second, parameter combinations are sampled with a seeded Latin hypercube rather than independent uniforms, which stratifies every marginal and fills the box more evenly at a given n ; this matters most in the 10-dimensional barrier problem. Third, the train/test split is random rather than chronological. The chronological rule we follow elsewhere in this homework exists to stop information flowing backwards in time; here the rows are independent draws from a fixed parameter box with no time ordering, so a random 80/20 split with a fixed seed is the correct protocol.

2.2 Ground truth A: CRR binomial tree for American options

The CRR tree (Cox, Ross, and Rubinstein 1979), set out in Lecture 9 and in Arratia (2014, ch. 5, Algorithm 5.1), divides the maturity into N steps of length $\Delta t = T/N$, over which the price moves by u or d with risk-neutral up probability p :

$$u = e^{\sigma\sqrt{\Delta t}}, \quad d = 1/u, \quad p = \frac{e^{(r-q)\Delta t} - d}{u - d},$$

where the course's p is extended with a continuous dividend yield q , under which the risk-neutral stock grows at $r - q$; the assignment lists q among the inputs to vary, and $q > 0$ also makes early exercise of the *call* possible, so both American contracts genuinely differ from their European counterparts. At the leaves the option is worth its payoff $\max(z(S - K), 0)$, with $z = \pm 1$ for call/put, and stepping backwards each node takes the maximum of the discounted expectation of its successors and immediate exercise,

$$V_{i,j} = \max \left(e^{-r\Delta t} (p V_{i+1,j+1} + (1-p) V_{i,j+1}), z(S_{i,j} - K) \right),$$

so the root value prices the American contract at $O(N^2)$ cost per option.

Validation and step count. Before the tree labels any data it passes three checks. With early exercise off it converges to the Black-Scholes-Merton price, as CRR must (at $N=4000$ the gap on a reference contract is 5×10^{-5} , with the usual odd/even oscillation on the way); the American put exceeds the European put (early-exercise premium 0.50 on the reference contract); and with $q = 0$ the American call equals the European call to a tenth of a cent, since early exercise of a call on a non-dividend stock is never optimal. A convergence study then fixes $N = 1000$ for dataset generation: the American put price at $N=1000$ is within 3×10^{-4} of an $N=8000$ reference while costing about 13 ms per option, so the labels carry a discretisation error three orders of magnitude below the price scale and can be treated as exact for the regression task.

Dataset A. We draw $n = 1500$ Latin hypercube points over the ranges in Table 4, near the top of the 500 to 2000 band the assignment allows; the binding constraint is not the tree (the 3000 prices take 37 seconds) but the GPR fit, whose cost grows with the cube of the training size. Each draw is priced twice, once as an American put and once as an American call, so the two contracts share one design matrix and the learnability of the two surfaces can be compared on equal terms. Prices average in the mid-teens of dollars with maxima near 54 where deep in-the-money meets high volatility and long maturity.

Table 4: Input ranges for the two synthetic datasets. Spot is fixed at $S_0 = 100$ throughout, so K and B are read as percentages of spot; maturities are in years.

Input	Range	Rationale
<i>Shared contract inputs</i>		
K	70 to 130	Deep ITM to deep OTM in both directions
T	0.1 to 2.0 (A); 0.25 to 2.0 (B)	Liquid maturities; B needs enough monitoring dates
r	0.00 to 0.10	Zero-rate world up to a high-rate regime
q	0.00 to 0.06	No dividends up to a generous yield
<i>Dataset A: American options, binomial tree</i>		
σ	0.10 to 0.60	Calm large-cap vol up to crisis vol
<i>Dataset B: down-and-out call, Heston Monte Carlo</i>		
B	60 to 95	Below S_0 by construction; barely binding to near-certain knock-out
ν_0	0.01 to 0.16	Initial variance (spot vol 10% to 40%)
θ	0.01 to 0.16	Long-run variance, same band
κ	0.5 to 5.0	Mean-reversion half-life of 17 months down to 7 weeks
η	0.1 to 0.8	Mild to violent vol-of-vol
ρ	-0.90 to 0.00	Equity leverage effect: spot down, vol up

2.3 Ground truth B: Heston Monte Carlo for the down-and-out call

The contract pays $\max(S_T - K, 0)$ unless the underlying touched or crossed the barrier $B < S_0$ at any monitoring date before T , in which case it pays nothing. The payoff depends on the whole path, so simulation is the natural pricer. Under Heston (Lecture 10),

$$dS_t = (r - q) S_t dt + \sqrt{\nu_t} S_t dW_t^S, \quad d\nu_t = \kappa(\theta - \nu_t) dt + \eta\sqrt{\nu_t} dW_t^\nu,$$

with $\text{Cov}[dW_t^S, dW_t^\nu] = \rho dt$, and we simulate with the Euler scheme of the L10 slides (equations 10 to 13): with $Z^\nu \sim N(0, 1)$ and $Z^S = \rho Z^\nu + \sqrt{1 - \rho^2} \epsilon$,

$$\nu_{t+\Delta t} = \nu_t + \kappa(\theta - \nu_t^+) \Delta t + \eta \sqrt{\nu_t^+ \Delta t} Z^\nu, \quad S_{t+\Delta t} = S_t \exp \left((r - q - \frac{1}{2} \nu_t^+) \Delta t + \sqrt{\nu_t^+ \Delta t} Z^S \right),$$

where $\nu^+ = \max(\nu, 0)$ is the full-truncation fix of Lord, Koekkoek, and Dijk (2010) for the negative variances the Euler step can produce, and the exponential price step keeps S positive. The barrier is monitored at every step with $\Delta t = 1/252$, i.e. a contract whose knock-out is checked at daily closes. Each price uses 20,000 paths and we record alongside

it the Monte Carlo standard error of the discounted average payoff, which is the noise level of the label and is used repeatedly below.

Cost, and one engineering device. Written as the explicit Lab 10 style loop over paths and steps, one 20,000-path price takes about 21 seconds, so a 1000-row dataset would need around six hours; this is precisely the time-consuming premise of the question. Vectorising the same scheme across paths (the Python loop runs only over time steps; nothing statistical changes) brings one price to 0.34 seconds, a factor of about 63. The loop and vectorised estimators agree within Monte Carlo error on a common contract (1.5 standard errors apart on independent draws). This path-vectorisation is deliberately the only acceleration used here; estimator-level improvements to the Euler MC are the subject of Question 3 and are kept out of this answer.

Validation. The simulator passes three checks of increasing strictness, none of which reuse its own code. (i) Our semi-analytic Heston call, the characteristic-function price of Heston (1993) in the trap-free formulation of Albrecher et al. (2007), collapses to Black-Scholes when $\eta \rightarrow 0$ and $\nu_0 = \theta$, matching to 1×10^{-4} , which validates the reference formula itself. (ii) With the barrier off, the simulated European Heston call matches the semi-analytic price to -0.7 standard errors, validating drift, diffusion, correlation and discounting. (iii) In the constant-volatility limit the simulated barrier price (9.832 ± 0.087) sits 0.9 standard errors from the Reiner-Rubinstein closed form (Reiner and Rubinstein 1991) once the Broadie-Glasserman-Kou correction (Broadie, Glasserman, and Kou 1997), with $\beta \approx 0.5826$, translates the continuous-barrier formula (9.580) to daily monitoring (9.754), validating the knock-out logic. The third check is a small lesson in itself: daily and continuous monitoring differ by about 25 cents here, roughly three standard errors, so the monitoring convention is economically real and our engine prices the daily-monitored contract it claims to price.

Dataset B. We draw $n = 1000$ Latin hypercube points over the ten inputs in Table 4; we vary the barrier B itself on top of the assignment’s list, since a surrogate that cannot move the defining parameter of the contract would be of little use. Generation takes 4.4 minutes at 261 ms per option, the longest single computation in the pipeline. We sample the Heston box freely, so 29.8% of rows violate the Feller condition $2\kappa\theta \geq \eta^2$; that is realistic (calibrated equity parameters violate it routinely) and is exactly what full truncation is for. Figure 4 shows the labels and their noise: prices are right-skewed with a long expensive tail and only 0.6% nearly dead contracts, and standard errors centre around 12 cents, largest for expensive contracts because payoff variance scales with the payoff. That average error is the floor below which no RMSE measured against these labels can fall.

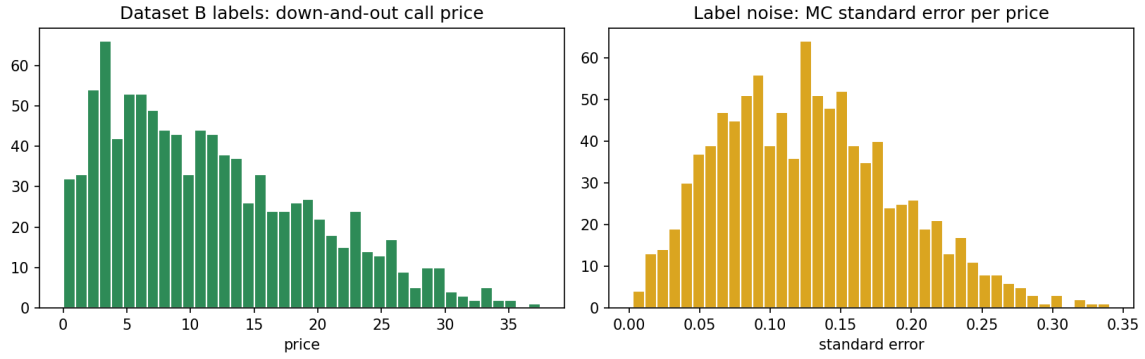


Figure 4: Dataset B: distribution of the 1000 down-and-out call prices (left) and of their Monte Carlo standard errors (right). The labels are noisy measurements of the pricing function, with an average noise of about 13 cents, and this noise level reappears in the GPR analysis below.

2.4 Learning the pricing functions

Protocol. Both models see identical data: an 80/20 random split (1200/300 rows for dataset A, 800/200 for B) and features standardised by a scaler fitted on the training portion only. Accuracy is reported on the held-out test set as RMSE and MAE in dollars, R^2 , the maximum absolute error, and the NRMSE used throughout the course labs, $\text{NRMSE} = \sqrt{\text{SSR}/((n-1)\text{Var}(y))}$, the fraction of the target’s standard deviation surviving as error. Because a slice of each label set sits near zero (the deep out-of-the-money, short-dated, low-volatility corner where the option is almost worthless), we read the absolute errors RMSE and MAE as the headline figures and treat ratio-based errors with care, since dividing by a near-zero price inflates them without signalling a worse model.

GPR. We use an anisotropic Matern $\nu = 5/2$ kernel with one length-scale per input (automatic relevance determination, ARD) and `normalize_y`, with two optimiser restarts. Matern is preferred to the smoother RBF because neither surface is analytic: the American early-exercise boundary and the barrier knock-out both leave kinked regions. The noise treatment is matched to the labels, which is the one deliberate deviation from the lab convention of a fixed nugget $\alpha = 10^{-2}$: that convention is built for noisy market data, and on our *deterministic* tree labels it blurs an exact surface (we measured a factor of six in test RMSE), so dataset A uses a numerical jitter $\alpha = 10^{-8}$ instead. For dataset B the labels really are noisy, so the kernel adds a `WhiteKernel` whose level is learned from the data, and this buys a self-test described below.

Neural network. A feed-forward `MLPRegressor` (the NNet family of Lab 6) with standardised targets, ReLU activations, Adam, and early stopping, tuned by 3-fold cross-validated grid search over hidden layouts $\{(32, 32), (64, 64), (64, 64, 64), (128, 128)\}$ and L2 penalties $\{10^{-4}, 10^{-3}\}$. The widest layout (128, 128) won on all three targets, consis-

tent with the targets being smooth functions where capacity is cheap and early stopping controls overfitting.

Accuracy. Table 5 reports the held-out metrics for all three targets and both models, and Figure 7 summarises them. Both families learn both pricing functions with $R^2 \geq 0.985$, and GPR is the stronger model throughout: on the American contracts its RMSE is below ten cents (MAE around four cents) on prices whose standard deviation exceeds ten dollars, an NRMSE below 0.01, and four to five times better than the tuned network. Residual diagnostics show no trend in any input for either model; for the American put the only structure is a mildly wider scatter at high strikes and low volatilities, exactly where the early-exercise kink is sharpest, and even the single worst test point is off by less than 75 cents on a surface spanning more than fifty dollars.

Table 5: Held-out test accuracy (dollars where dimensional). Dataset A: 300 test rows against binomial-tree truth. Dataset B: 200 test rows against Heston MC truth, whose labels carry an average standard error of \$0.125; the measured RMSE mixes model error with this label noise.

Target	Model	RMSE	MAE	R^2	NRMSE	max error
American put	GPR	0.088	0.040	0.9999	0.008	0.73
	NN	0.428	0.300	0.9985	0.039	2.30
American call	GPR	0.069	0.037	1.0000	0.007	0.41
	NN	0.320	0.233	0.9990	0.032	1.50
Barrier DO call	GPR	0.320	0.232	0.9985	0.039	1.78
	NN	0.979	0.733	0.9859	0.119	4.23

What the GPR learned, beyond accuracy. Two by-products of the GP are worth reporting because they show the model has learned structure rather than memorised points. First, the fitted ARD length-scales rank the inputs by influence exactly as option economics would: for the American contracts the strike and the volatility carry the shortest scales (strongest influence), maturity sits in the middle, and the rates r and q are gentlest; for the barrier contract the shortest scales belong to K and B , the two prices the spot is measured against, with the variance levels in the middle and η and ρ longest, second-order shape effects for a contract dominated by where the spot sits relative to barrier and strike. Second, the learned `WhiteKernel` noise on dataset B comes out at 14.2 cents against a recorded average Monte Carlo standard error of 12.8 cents on the same training rows: two estimates of the same quantity by completely independent routes (one statistical, one computed path-by-path at generation time) landing within a cent and a half. The match cannot be exact, since the true noise is heteroskedastic while the `WhiteKernel` fits one average level, but it is strong evidence the GP separated signal

from noise rather than overfitting it.

Figures 5 and 6 visualise the learned functions directly, slicing price against strike at otherwise fixed parameters and overlaying freshly computed ground truth never seen in training. On the American slice the GPR mean passes through all 61 fresh tree prices with a largest gap of five cents and a 95% band too tight to see, while the NN reproduces the shape but sags by up to about a dollar through the middle of the range, the same gap as in Table 5. On the barrier slice all 25 fresh Monte Carlo prices fall inside the GPR predictive band, and the band width is comparable to the error bars it is meant to cover, which is what a correctly calibrated noise-aware GP should produce: each black dot cost a quarter of a second of simulation, while the curve through them was evaluated at every strike in under two milliseconds, with honest uncertainty attached.

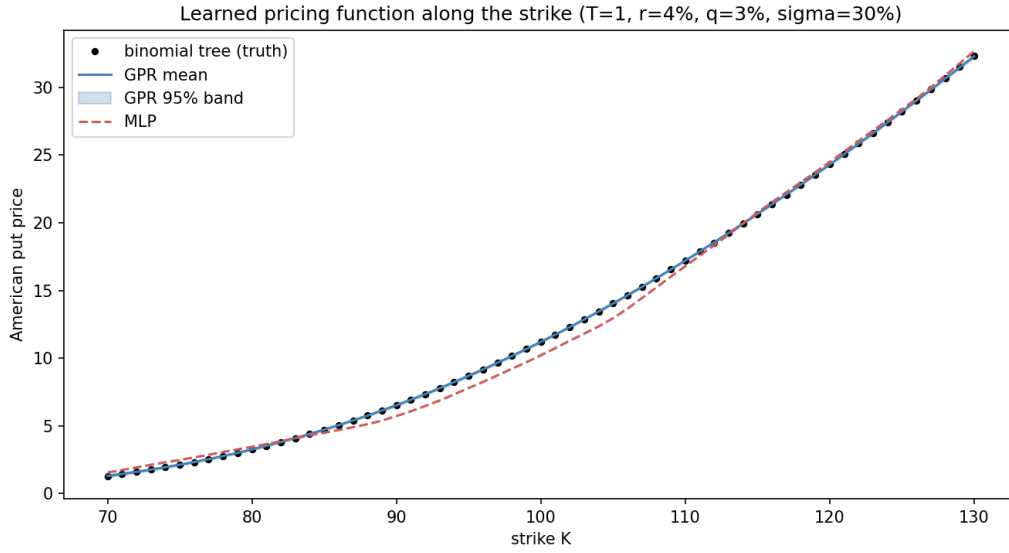


Figure 5: Learned American put pricing function along the strike ($T=1$, $r=4\%$, $q=3\%$, $\sigma=30\%$). Black dots are fresh binomial-tree prices computed at plot time; the GPR mean (with its barely visible 95% band) passes through all of them, the NN tracks the shape with a visible sag around the money.

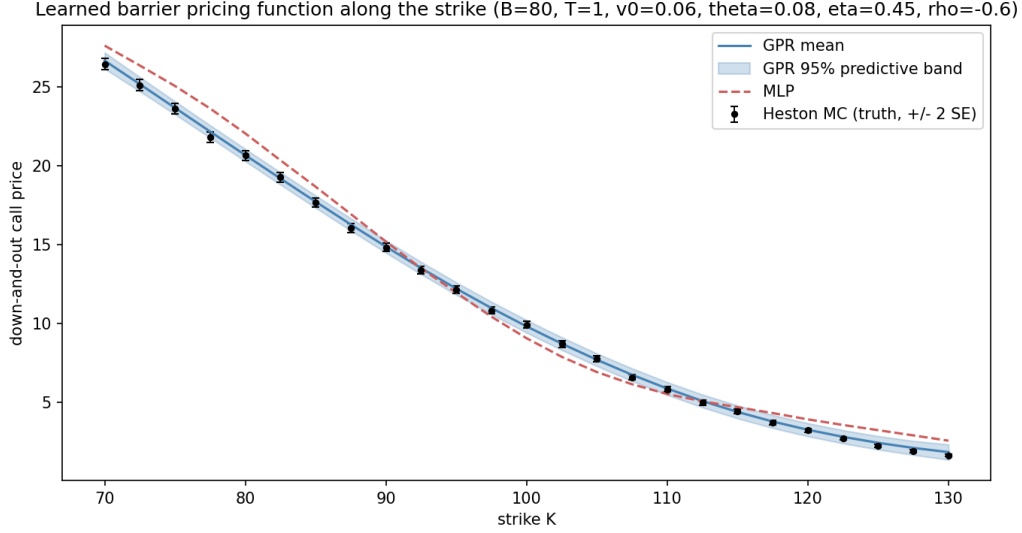


Figure 6: Learned barrier pricing function along the strike ($B=80, T=1, \nu_0=0.06, \theta=0.08, \eta=0.45, \rho=-0.6$). Black dots are fresh Heston MC prices with ± 2 standard error bars; all 25 fall inside the GPR 95% predictive band, whose width includes the learned label noise.

Reading the barrier errors. The barrier surrogate is visibly less accurate than the American one, and the decomposition matters for what to do about it. The GPR test RMSE of \$0.32 is about 2.6 times the average label noise of \$0.125; subtracting the noise floor in quadrature leaves roughly \$0.29 of genuine approximation error. So although the labels are noisy, the binding constraint is the harder function: a 10-dimensional surface with a kinked knock-out region, learned from 800 points. The practical implication is that more training rows would still buy accuracy, while more paths per label only starts to pay once model error approaches the noise floor. The few residuals beyond a dollar belong to contracts hugging the barrier, the steepest and least smooth corner of the surface.

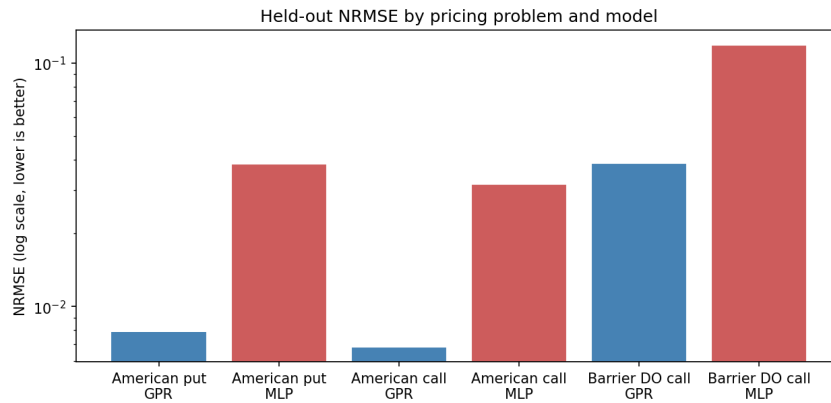


Figure 7: Held-out NRMSE by pricing problem and model (log scale). GPR beats the network on every target; both degrade moving from noiseless 5-dimensional tree labels to noisy 10-dimensional Monte Carlo labels.

2.5 Computational speedup

Table 6 times every pricer per option (engines per call; surrogates by batch prediction over the test set divided by its size, which is how a surrogate is used) and separates the one-off costs of dataset generation and training. The break-even column divides the one-off cost by the per-option saving: the number of pricings after which the surrogate has repaid its own construction.

Table 6: Per-option pricing cost, speedup over the ground-truth engine, one-off cost (dataset generation attributable to the target, plus model fitting), and break-even pricing count. The tree uses $N = 1000$ steps; the Monte Carlo uses 20,000 paths with daily steps. Engine times are clean per-call timings; the per-option figure during dataset generation is slightly higher once per-row overhead is included.

Problem	Pricer	ms/option	Speedup	One-off (s)	Break-even
American	binomial tree	13.0	1	–	–
	GPR surrogate	0.068	190	149	~11,500
	NN surrogate	0.005	2454	65	~5,000
Barrier	Euler MC	236.4	1	–	–
	GPR surrogate	0.070	3391	315	~1,300
	NN surrogate	0.010	23358	300	~1,300

The network prices an American option about 2,500 times faster than the tree and a barrier option more than 20,000 times faster than the Monte Carlo; GPR is roughly an order of magnitude slower than the network, because its prediction touches every training point through the kernel, but still beats the engines by factors of 190 and 3,400. The speedup is largest exactly where the engine is slowest, and there the economics are immediate: the barrier pipeline’s entire offline investment, around five minutes of simulation plus the fits, is repaid after about 1,300 pricings, which a single calibration loop or a risk-system revaluation of a book consumes at once. The honest counterpart is that the speed arrives together with label noise and problem difficulty: the fastest speedup belongs to the least precise surrogate, not because the regression is worse but because its ground truth is noisier and higher-dimensional.

2.6 Comparison of the two pipelines, and conclusions

Tree labels versus Monte Carlo labels. Both pipelines learn their pricing function with R^2 above 0.98, but the binomial pipeline is several times more accurate in NRMSE for the better model. The reasons are structural: noiseless smooth labels of a 5-dimensional function from 1200 examples, against noisy labels (13 cents average standard error) of a 10-dimensional kinked function from 800 examples. Of the three handicaps (noise floor,

dimension, surface roughness), the decomposition above shows dimension and roughness dominate, so the gap would close with more rows, not with more simulation paths.

GPR versus the neural network. At these dataset sizes GPR wins on accuracy on every target, and it provides two things the network does not: calibrated uncertainty bands (Figure 6) and an internally estimated label-noise level that independently matched the known Monte Carlo standard error. The network wins on prediction speed by roughly an order of magnitude and on scalability, since GP training is cubic in n and GP prediction cost grows with n , while the network’s training scales gently and its prediction cost is flat. At the 5,000 to 10,000-row datasets the assignment calls ideal, the balance tilts towards the network; at our sizes the recommendation is GPR where accuracy and risk-awareness matter, the network where prediction throughput dominates.

Limitations. The surrogates are trusted only inside the sampled box of Table 4; extrapolation beyond it is unprotected, although the GPR band at least widens there. The barrier labels carry Euler discretisation bias at $\Delta t = 1/252$ on top of Monte Carlo noise, and the surrogate prices a daily-monitored contract by construction, a contract definition rather than an approximation, as the validation against the Broadie-Glasserman-Kou correction made explicit. The spot is normalised to $S_0 = 100$ by homogeneity. Finally, the only acceleration applied to the Monte Carlo here is vectorisation across paths; variance-reduction and other estimator-level improvements are developed in our answer to Question 3, where they belong.

3 Improvements to the Euler MC

We propose and implement five improvements to the Euler Monte Carlo engine of our solution to Question 2, the simulator that generated the barrier dataset there (Heston dynamics, full-truncation Euler steps, daily barrier monitoring, 20,000 paths per price). “Faster” can mean two different things, and we implement both kinds: *engineering* improvements that compute the identical estimator in less wall-clock time per path (path-vectorisation, just-in-time compilation with Numba, multithreading), and *statistical* improvements that reach the same accuracy from fewer paths (antithetic variates, and a control variate that reuses the semi-analytic Heston pricer from our Q2 validation as its known mean). A third conceivable lever, enlarging the time step, is shown below to be unavailable for this product. Everything is implemented in the submitted notebook `ML.PS2.Q3.ipynb`, on the same benchmark contract as Q2 (at the money, one year, $B = 85$), and the section closes by pointing the improvements back at the Q2 task itself: regenerating rows of the actual dataset at matched accuracy.

3.1 One yardstick for every improvement

A Monte Carlo price is a sample mean over M independent units (paths, or path-pairs) with standard error $\sqrt{v/M}$ and cost cM , where v is the per-unit variance and c the per-unit time. Reaching a target accuracy ε therefore costs

$$T(\varepsilon) = c \cdot \frac{v}{\varepsilon^2},$$

so the figure of merit for any estimator is the product $c \cdot v$, its work-normalised variance, and the equal-accuracy speedup of method 2 over method 1 is $(c_1 v_1)/(c_2 v_2)$ (the efficiency comparison of Glasserman 2004, ch. 4). Engineering lowers c , variance reduction lowers v , and the two multiply. The metric also keeps us honest in both directions: a rewrite that is slower per path counts against, and a trick that doubles per-path cost must at least halve the variance to break even. All variants below print their price and standard error on the benchmark contract, and all agree within Monte Carlo error (the largest discrepancy in the notebook is below one standard error), so the comparison is between estimators of the same quantity.

3.2 Engineering: the same estimator, computed faster

Three rungs sit above the original Lab 10 style pure-Python loop, which costs about 1 ms per path on our machine. The NumPy path-vectorised engine (the device already used in Q2, where it was the single permitted acceleration) advances all paths simultaneously and is a factor of about 50 cheaper per path. A Numba just-in-time compiled per-path loop removes NumPy’s remaining overheads, the dozen temporary 100,000-element arrays allocated at every one of the 252 steps, and adds something vectorised code cannot do: a path that hits the barrier can **break** immediately instead of being carried dead through every remaining step.

The measured outcome contains an instructive negative result: the *serial* JIT loop is slower than the vectorised engine (0.8 times its efficiency), because per-draw calls to Numba’s random generator cost more than the removed overheads save, and only a minority of paths knock out early enough for the break to pay. Compilation is not a magic multiplier over already-vectorised code. The parallel JIT kernel is the genuine engineering win, roughly threefold over the vectorised engine on eight threads. One caveat is verified in the notebook rather than assumed: Numba’s parallel random streams make the parallel kernels statistically but not bit-for-bit reproducible across runs, so for dataset generation, where exact reproducibility matters, we keep the seeded NumPy engines and treat parallel JIT as the throughput option.

3.3 Statistics: the same accuracy, from fewer paths

Antithetic variates. Simulating paths in pairs that share the same normal draws with flipped signs, and averaging each pair’s payoffs, is unbiased and reduces variance whenever the payoff is broadly monotone in the shocks (Glasserman 2004, sec. 4.2), which a down-and-out call is: up-shocks raise it, down-shocks knock it out. Measured on the benchmark contract, the per-equal-path-budget variance falls by 1.9, and since each pair consumes only one set of random numbers for two paths, the per-pair cost is slightly below two path costs. Net effect: about 2.7 times the baseline’s efficiency.

A control variate from the Q2 validation apparatus. The strongest single improvement adjusts the estimator with a correlated quantity of known mean,

$$\bar{Y}_{cv} = \bar{Y} - \beta (\bar{X} - \mathbb{E}[X]), \quad \beta^* = \frac{\text{Cov}(Y, X)}{\text{Var}(X)}, \quad \frac{\text{Var}[\bar{Y}_{cv}]}{\text{Var}[\bar{Y}]} = 1 - \rho_{YX}^2,$$

and our solution to Q2 supplies the ideal X ready-made: the *European* call payoff on the same simulated paths. It costs nothing extra (the terminal prices are already in hand), it is highly correlated with the barrier payoff because the two are identical on every path that does not knock out, and its exact expectation under Heston is known semi-analytically from the characteristic-function pricer we built to validate the Q2 engine (Heston 1993; Albrecher et al. 2007). On the benchmark contract the measured correlation is 0.96 and the variance falls by a factor of 11.7 at essentially unchanged cost. The coefficient β is estimated from the same sample, a bias of order $1/M$ that is negligible here (Glasserman 2004, sec. 4.1). One structural cost is worth noting: because the control variate reads the terminal price S_T on every path, an estimator that carries it cannot use the JIT early knock-out exit and must simulate each path to maturity, so the engineering and statistical rungs do not stack costlessly; on this contract the order-of-magnitude variance reduction far outweighs the handful of paths not cut short. One further nuance is disclosed and then tested: the analytic mean is exact for Heston while the simulated X carries the Euler scheme’s $O(\Delta t)$ weak bias, so the adjustment shifts the estimand by β times that small gap; the dataset experiment below measures the end-to-end effect and finds it statistically invisible at daily steps.

Antithetic pairing and the control variate attack different noise (sign asymmetry versus level), so they stack, and both being estimator-level they also compose with the engineering rungs. Table 7 and Figure 8 put all eight variants on the common yardstick: statistics alone buys an order of magnitude over the Q2 engine, engineering alone a factor of about three, and the top rung, antithetic plus control variate inside the parallel JIT kernel, is roughly forty times the Q2 production engine and three orders of magnitude above the original loop.

Table 7: The efficiency ladder on the benchmark contract ($B = 85$, measured timings on our machine, 100,000 path budget per run). Units are paths, or antithetic pairs where applicable (one pair simulates two paths from one set of draws). Speedups are equal-accuracy ratios of $c \cdot v$.

Method	Time/unit (μ s)	Var/unit	vs loop	vs Q2 engine
Pure Python loop (Lab 10 style)	1039	152.4	1	0.02
NumPy vectorised (Q2 engine)	20.0	152.4	52	1
Numba JIT serial	25.6	153.2	40	0.8
Numba JIT parallel (8 threads)	5.9	152.1	177	3.4
Vectorised + antithetic	27.1	41.0	143	2.7
Vectorised + control variate	16.9	13.1	718	13.8
Vectorised + antithetic + CV	27.1	6.8	865	16.7
Numba parallel + antithetic + CV	12.0	6.6	1993	38.4

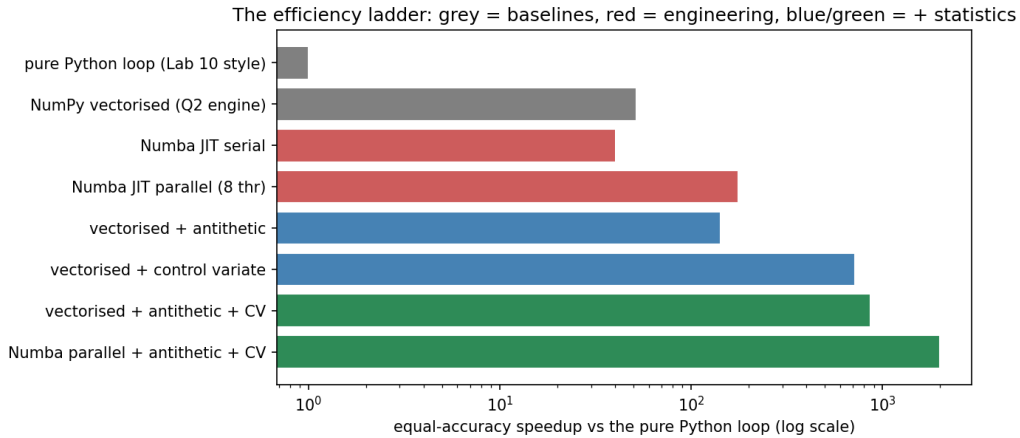


Figure 8: Equal-accuracy speedup over the pure Python loop (log scale). Grey bars are the Q2 baselines, red the engineering rungs, blue and green the statistical rungs and their combinations. Engineering and statistics multiply; the serial JIT bar sitting below the vectorised bar is the reported negative result.

3.4 When the gains shrink, and the lever we refuse to pull

The control variate's power is the correlation ρ_{YX} , which is a property of the contract: high when knock-out is rare, lower as the barrier rises towards the spot. Repeating the headline measurement across barrier levels (Table 8) shows the combined estimator beating the Q2 engine by three orders of magnitude when the barrier is far (the correlation is then 0.999 and almost all noise is regressed away), falling to about three times in the hardest corner of the Q2 parameter box, where three quarters of paths knock out. Two practical readings: the improvements never hurt, and the per-contract gain is predictable from a cheap pilot estimate of ρ_{YX} , which the adaptive scheme below exploits.

Table 8: Robustness of the combined (antithetic + control variate) estimator across barrier levels, all other inputs at benchmark values. The speedup is the equal-accuracy ratio against the Q2 vectorised engine at the same B .

B	P(knock-out)	corr(Y, X)	Var/pair (anti+CV)	Speedup vs Q2 engine
65	0.10	0.999	0.04	~ 2725
80	0.32	0.970	2.24	~ 52
90	0.58	0.820	15.3	~ 7.3
95	0.75	0.637	28.2	~ 3.2

For a European payoff one could also trade weak bias for speed by enlarging Δt . For this contract one cannot, and we demonstrate it rather than assert it: the step grid *is* the monitoring grid, so coarsening it prices a different option, not the same option less accurately. Priced with the combined estimator (whose tight error bars are themselves a first practical dividend of the variance reduction), the benchmark contract is worth 8.745 ± 0.011 monitored every other day, 8.685 ± 0.012 monitored daily (the contract), and 8.652 ± 0.012 monitored twice daily: each halving of the monitoring interval moves the price by two to four standard errors, in the direction more checks, more knock-outs, lower value, the same $\sqrt{\Delta t}$ effect the Broadie-Glasserman-Kou correction (Broadie, Glasserman, and Kou 1997) quantified in our Q2 validation. For a daily-monitored product, 252 steps per year is the contract, not a numerical knob, and all speed must come from the path dimension, which is what the ladder delivered. Coarser stepping would be legitimate only for a contract monitored less often than daily, or between monitoring dates under a scheme that can take long exact steps (see the Andersen route below).

3.5 The payoff in our solution to Q2: regenerating dataset B

Improvements only matter if they accelerate the task Q2 actually performs, so we close the loop. We re-price a random 100 of the 1000 rows of the Q2 barrier dataset with the combined estimator at *matched accuracy*, targeting each row’s own recorded standard error. The scheme is adaptive: a 1000-pair pilot estimates the row’s control coefficient and post-CV variance, the budget needed for the row’s target follows from $M = v/\varepsilon^2$, extra pairs are simulated and pooled if needed, and the analytic control mean is computed per row. We use the seeded NumPy implementation so the regenerated labels are exactly reproducible, and all overheads (pilot, analytic mean, bookkeeping) are charged to the measured time.

The outcome: the 100 rows re-price in 6.8 seconds against roughly 26 seconds of Q2 engine time, a wall-clock factor of 3.8, with an average budget of about 1,550 pairs per row against Q2’s 20,000 paths. The accuracy is in fact better than matched (mean achieved standard

error \$0.064 against the \$0.110 target, because the pilot floor over-serves easy rows), so the wall-clock factor is conservative; scaled up, the full dataset that took 4.4 minutes in Q2 regenerates in about one. The gap to the asymptotic factor of Table 7 is per-row fixed overheads (pilot, control mean, short-vector calls), which amortise for bigger jobs and shrink further under the parallel kernel.

The comparison doubles as an end-to-end correctness test of every piece at once. With independent random numbers, the standardised differences

$$z_i = \frac{\hat{p}_i^{new} - \hat{p}_i^{Q2}}{\sqrt{SE_{new,i}^2 + SE_{Q2,i}^2}}$$

should be standard normal; Figure 9 shows they are (95% inside ± 2 , standard deviation 0.97). Their mean, $+0.05 \pm 0.10$, is the empirical verdict on the control-variate nuance: no detectable bias at daily steps, exactly as argued.

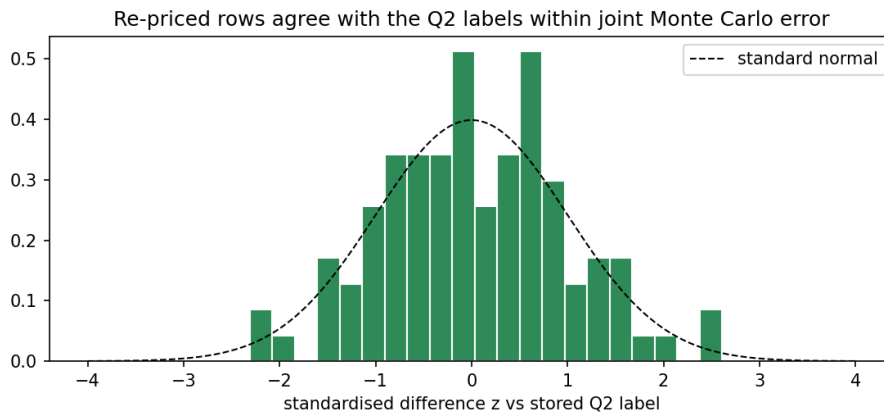


Figure 9: Standardised differences between the re-priced labels and the stored Q2 labels for 100 dataset rows, against the standard normal density. Agreement within joint Monte Carlo error validates the combined estimator end to end, and the mean of $+0.05$ shows the control variate’s $O(\Delta t)$ estimand shift is invisible at this accuracy.

Further routes and closing remarks. Routes we propose but do not implement, each pluggable into the same engine and yardstick: quasi-Monte Carlo with Sobol points (ideally with a Brownian-bridge path construction so the low-dimensional accuracy of the point set is spent on the most important path features); importance sampling, which pays most for far out-of-the-money contracts where most paths are wasted; multilevel Monte Carlo (Giles 2008), whose standard form targets discretisation bias and is therefore constrained here by the contractual monitoring grid; the Andersen (2008) quadratic-exponential scheme, which simulates the Heston variance nearly exactly and becomes valuable when monitoring is coarser than daily; and, on the engineering side, batching dataset rows into one fused kernel, mixed precision, and GPUs. The honesty items from

the implementation stand: serial JIT alone is a negative result, the parallel kernels trade bit-reproducibility for throughput, the control variate’s strength is contract-dependent in a predictable way, and its small estimand shift was measured and found invisible. Within those bounds, the combination delivered is roughly forty times the efficiency of the engine we shipped in Q2, and three orders of magnitude over the loop the course started from.

A Trade-by-trade log of the headline strategies

The tables below list every round trip taken by each headline strategy over the full sample, with entry and exit dates, the prices, and the return earned. One row is one holding period at a constant position, from opening to closing. The entry price is the close on the day the position was opened, which is the day before it becomes active because the signal is shifted one day. The exit price is the close on the last day the position was held. For the single-stock strategy the return is the price move signed by the direction held. For the pairs strategy it is the spread move, the long leg return minus the short leg return.

The Lab 6 trade count reported in Section 1 counts position changes rather than round trips, so it does not equal the number of rows here: the single-stock headline shows 7 round trips against 12 counted position changes, and the pairs headline shows 5 round trips against 6 position changes per leg (12 across the two legs). Because the per-trade figures are price-based round trips, their sums (62.91% single-stock, 69.95% pairs) do not exactly match the additive, cost-charged cumulative return in Section 1, which sums daily simple returns and charges the 1% cost as one lump. The Section 1 metrics remain the headline performance figures. These tables show when each position was taken, at what price, and what it earned.

A.1 Single-stock headline

Table 9: Trade log, single-stock headline: HSBC positiveP, contrarian, long-short, $(n, k, q) = (20, 2.0, 0.85)$. Returns in percent, signed by direction. Total cost at 1% per position change: 12.00%.

Entry	Exit	Days	Direction	Entry price	Exit price	Return (%)
2018-06-11	2018-10-29	140	Short	43.83	36.55	+16.61
2018-11-01	2019-02-20	111	Short	37.81	38.33	-1.37
2019-04-03	2019-10-04	184	Short	39.70	35.63	+10.26
2019-10-04	2019-10-22	18	Long	35.63	38.06	+6.82
2019-10-22	2020-02-18	119	Short	38.06	34.77	+8.65
2020-03-31	2020-04-01	1	Short	28.01	25.14	+10.25
2020-04-08	2020-05-22	44	Short	26.01	22.97	+11.69
Sum of round-trip returns						+62.91

A.2 Pairs headline

Table 10: Trade log, pairs headline: JPM/HSBC positiveP spread, momentum, long-short, $(n, k) = (20, 2.0)$. The Long / Short column gives the long leg then the short leg. Spread is the long leg return minus the short leg return, in percent. Total cost at 1% per change across two legs: 12.00%.

Entry	Exit	Long / Short	JPM price		HSBC price		Spread (%)
			In	Out	In	Out	
2018-02-13	2019-10-24	JPM / HSBC	105.36	122.95	44.59	38.30	+30.80
2019-10-24	2019-11-18	HSBC / JPM	122.95	128.45	38.30	36.14	-10.10
2019-11-18	2020-02-11	JPM / HSBC	128.45	136.58	36.14	37.12	+3.62
2020-02-11	2020-03-20	HSBC / JPM	136.58	82.64	37.12	28.81	+17.10
2020-03-20	2020-05-22	JPM / HSBC	82.64	89.47	28.81	22.97	+28.53
Sum of round-trip spread returns							+69.95

References

- Albrecher, Hansjörg, Philipp Mayer, Wim Schoutens, and Jurgen Tistaert (2007). “The Little Heston Trap”. In: *Wilmott Magazine*. January issue, pp. 83–92.
- Andersen, Leif (2008). “Simple and Efficient Simulation of the Heston Stochastic Volatility Model”. In: *Journal of Computational Finance* 11.3, pp. 1–42.
- Arratia, Argimiro (2014). *Computational Finance: An Introductory Course with R*. Vol. 1. Atlantis Studies in Computational Finance and Financial Engineering. Atlantis Press.
- Broadie, Mark, Paul Glasserman, and Steven Kou (1997). “A Continuity Correction for Discrete Barrier Options”. In: *Mathematical Finance* 7.4, pp. 325–349.
- Cox, John C., Stephen A. Ross, and Mark Rubinstein (1979). “Option Pricing: A Simplified Approach”. In: *Journal of Financial Economics* 7.3, pp. 229–263.
- Giles, Michael B. (2008). “Multilevel Monte Carlo Path Simulation”. In: *Operations Research* 56.3, pp. 607–617.
- Glasserman, Paul (2004). *Monte Carlo Methods in Financial Engineering*. Vol. 53. Stochastic Modelling and Applied Probability. New York: Springer.
- Heston, Steven L. (1993). “A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options”. In: *The Review of Financial Studies* 6.2, pp. 327–343.
- Lord, Roger, Remmert Koekkoek, and Dick van Dijk (2010). “A Comparison of Biased Simulation Schemes for Stochastic Volatility Models”. In: *Quantitative Finance* 10.2, pp. 177–194.
- Reiner, Eric and Mark Rubinstein (1991). “Breaking Down the Barriers”. In: *Risk* 4.8, pp. 28–35.